

SOFTWARE DESIGN DOCUMENT (SDD)

Utility Billing System

1. DOCUMENT CONTROL

Item	Details
Project Name	Utility Billing System
Version	1.0
Author	Bhavana Yeluri
Date	29/12/2025
Status	Final

2. PURPOSE OF THE DOCUMENT

This document describes the **system architecture, design decisions, component structure, APIs, data models, and non-functional aspects** of the Utility Billing System.

It is intended for:

- Developers
 - Reviewers
 - Interview discussions
 - Maintenance & enhancement planning
-

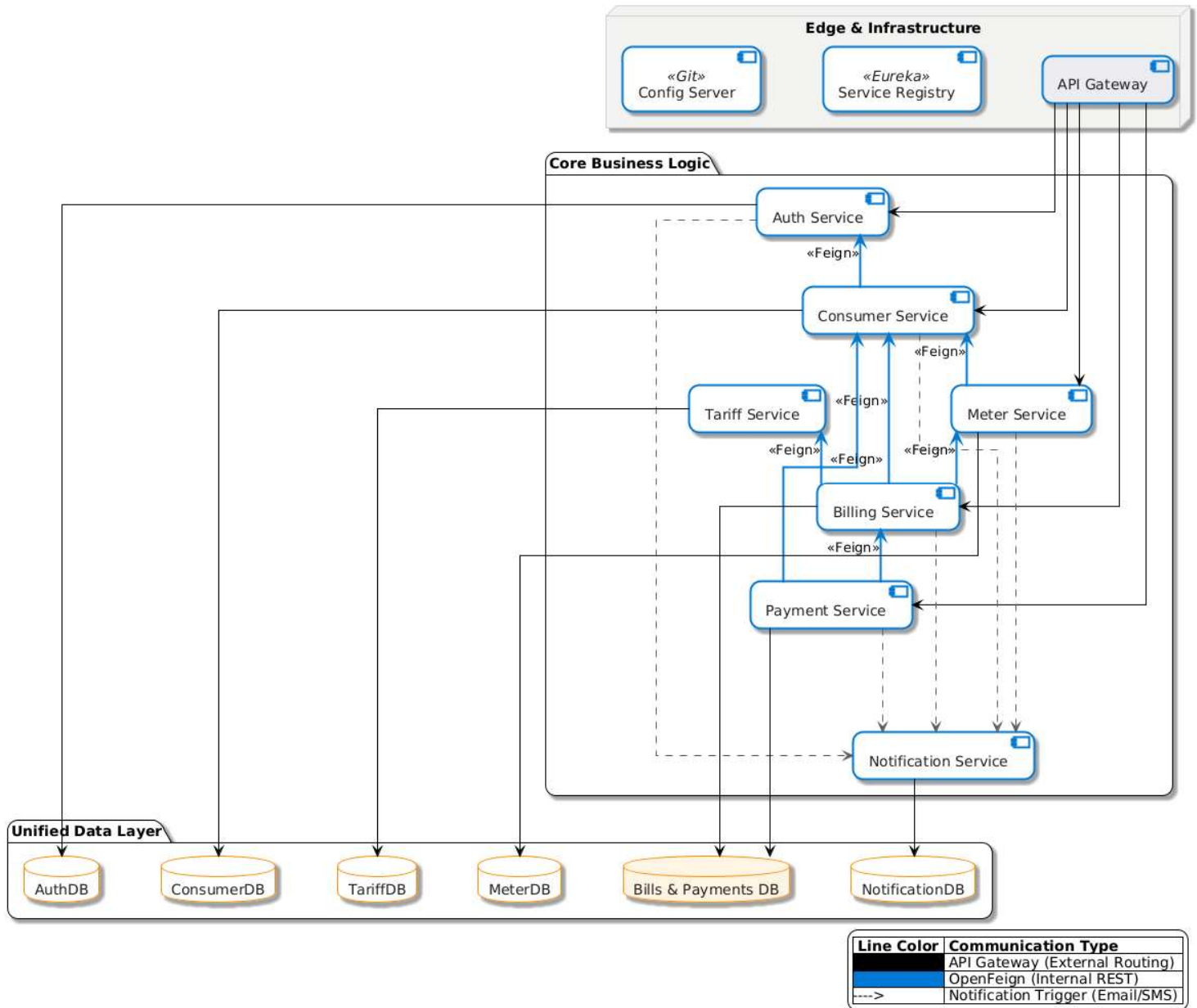
3. SYSTEM OVERVIEW

Business Objective

- **Centralized Utility Management** – Provide a single platform to manage multiple utilities, consumers, billing, and payments.
 - **Automated and Accurate Billing** – Generate timely and error-free bills based on consumption data.
 - **Secure Role-Based Access** – Enforce JWT-based authentication and controlled access for different user roles.
 - **Flexible Payment Processing** – Support both online and offline payment methods for user convenience.
 - **Real-Time Notifications and Transparency** – Notify users about bills, payments, and reminders to improve transparency.
 - **Scalable and Fault-Tolerant Architecture** – Use microservices, centralized configuration, and resilience mechanisms to ensure scalability and availability.
-

4. ARCHITECTURE OVERVIEW (HLD)

Utility Billing System - Microservices Architecture



Architecture Style

- Microservices Architecture
- REST-based communication
- Containerized deployment

Core Components

- **API Gateway** – Acts as the single entry point, handling request routing, JWT validation, and role-based access control.
 - **Service Registry (Eureka Server)** – Enables dynamic service discovery and load balancing among microservices.
 - **Config Server** – Provides centralized, version-controlled configuration management for all microservices.
 - **Auth Service** – Manages user authentication, authorization, and JWT token generation.
 - **Consumer Service** – Handles consumer onboarding, approval workflows, and consumer profile management.
 - **Meter Service** – Manages utility connection requests, meter assignment, and meter readings.
 - **Tariff Service** – Maintains tariff plans, slabs, taxes, and penalty configurations for utilities.
 - **Billing Service** – Generates utility bills based on meter readings, tariff rules, and consumption data.
 - **Payment Service** – Processes online and offline payments, verifies transactions, and generates invoices.
 - **Notification Service** – Sends system notifications such as emails and OTPs for approvals, payments, and alerts.
 - **Databases** – Maintains isolated MongoDB databases per service for scalability and data independence.
 - **Resilience and Fault Tolerance Layer** – Prevents cascading failures and ensures system stability.
-

5. TECHNOLOGY STACK

Backend

- Java 17
- Spring Boot
- Spring Web
- Spring Data MongoDB
- Spring Validation
- Open Feign
- Resilience 4j (Circuit Breaker)

Frontend

- Angular

Database

- MongoDB (one DB per microservice)

DevOps

- Docker
- Docker Compose
- Jenkins
- SonarQube

Testing

- JUnit 5
- Mockito
- Spring Boot Test

6. MICROSERVICES DESIGN

6.1 Auth Service

Responsibilities

- User registration and login
- JWT token generation and validation
- Role-based access control
- Forgot password and reset password flows

APIs

POST /auth/register

Request Payload

```
{
  "username": "john_doe",
  "email": "john.doe@gmail.com",
  "password": "Password@123",
  "role": "CONSUMER"
}
```

Response Payload

```
{
  "userId": "U101",
  "message": "User registered successfully"
}
```

POST /auth/login

Request Payload

```
{
  "username": "string",
  "password": "string"
}
```

Response Payload

```
{
  "token": "string"
}
```

POST /auth/forgot-password

Request Payload

```
{
```

```
    "email": "john.doe@gmail.com"
  }
Response Payload
{
  "message": "Password reset link sent to registered email"
}
```

POST /auth/reset-password

Request Payload

```
{
  "resetToken": "reset-token-xyz",
  "newPassword": "NewPassword@123"
}
```

Response Payload

```
{
  "message": "Password reset successful"
}
```

PUT /auth/change-password

Request Payload

```
{
  "oldPassword": "Password@123",
  "newPassword": "UpdatedPassword@123"
}
```

Response Payload

```
{
  "message": "Password updated successfully"
}
```

GET /auth/users

Response Payload

```
[
  {
    "userId": "U101",
    "username": "john_doe",
    "email": "john.doe@gmail.com",
    "role": "CONSUMER",
    "status": "ACTIVE"
  }
]
```

GET /auth/users/{userId}

Response Payload

```
{
  "userId": "U101",
  "username": "john_doe",
  "email": "john.doe@gmail.com",
  "role": "CONSUMER",
  "status": "ACTIVE"
}
```

DELETE /auth/users/{userId}

Response Payload

```
{
  "message": "User deleted successfully"
}
```

Database

- users collection
- password_reset_tokens collection

6.2 Consumer Service

Responsibilities

- Consumer registration and profile management
- Linking consumers to utility connections
- Managing connection status

APIs

POST /consumers

Request Payload

```
{
  "name": "Alice Johnson",
  "phone": "9876543210",
  "email": "alice@gmail.com",
  "address": "Hyderabad"
}
```

Response Payload

```
{
  "consumerId": "C201",
  "message": "Consumer registered successfully"
}
```

GET /consumers/{consumerId}

Response Payload

```
{
  "consumerId": "C201",
  "name": "Alice Johnson",
  "phone": "9876543210",
  "email": "alice@gmail.com",
  "address": "Hyderabad",
  "status": "ACTIVE"
}
```

GET /consumers

Response Payload

```
[
  {
    "consumerId": "C201",
    "name": "Alice Johnson",
    "status": "ACTIVE"
  }
]
```

PUT /consumers/{consumerId}

Request Payload

```
{
  "phone": "9999999999",
  "address": "Bangalore"
}
```

Response Payload

```
{
  "message": "Consumer updated successfully"
}
```

DELETE /consumers/{consumerId}

Response Payload

```
{
```

```
"message":"Consumer deactivated successfully"
}
```

POST /connections

Request Payload

```
{
  "consumerId":"C201",
  "utilityId":"U1",
  "meterNumber":"MTR1001",
  "tariffId":"T101"
}
```

Response Payload

```
{
  "connectionId":"CN301",
  "status":"ACTIVE"
}
```

GET /connections/{consumerId}

Response Payload

```
[
  {
    "connectionId":"CN301",
    "utilityId":"U1",
    "meterNumber":"MTR1001",
    "status":"ACTIVE"
  }
]
```

PUT /connections/{connectionId}

Request Payload

```
{
  "status":"DISCONNECTED"
}
```

Response Payload

```
{
  "message":"Connection status updated"
}
```

Database

- consumers collection
- connections collection

6.3 Utility Service

Responsibilities

- Manage utility types (Electricity, Water, Gas, etc.)
- Define billing units and billing cycles
- Manage tariff slabs, taxes, and fixed charges

APIs

POST /utilities

Request Payload

```
{
  "name":"ELECTRICITY",
  "unit":"kWh",

```

```
    "billingCycle":"MONTHLY"
  }
}
```

Response Payload

```
{
  "utilityId":"U1",
  "message":"Utility created successfully"
}
```

GET /utilities

Response Payload

```
[
  {
    "utilityId":"U1",
    "name":"ELECTRICITY",
    "unit":"kWh",
    "billingCycle":"MONTHLY"
  }
]
```

GET /utilities/{utilityId}

Response Payload

```
{
  "utilityId":"U1",
  "name":"ELECTRICITY",
  "unit":"kWh",
  "billingCycle":"MONTHLY"
}
```

PUT /utilities/{utilityId}

Request Payload

```
{
  "unit":"Units",
  "billingCycle":"MONTHLY"
}
```

Response Payload

```
{
  "message":"Utility updated successfully"
}
```

POST /tariffs

Request Payload

```
{
  "utilityId":"U1",
  "slabs":[
    {"min":0,"max":100,"rate":3.5},
    {"min":101,"max":300,"rate":5.0}
  ],
  "fixedCharge":120,
  "taxPercent":5
}
```

Response Payload

```
{
  "tariffId":"T101",
  "message":"Tariff created successfully"
}
```

GET /tariffs

Response Payload

```
[
  {
    "tariffId":"T101",
    "utilityId":"U1",
    "fixedCharge":120,
    "taxPercent":5
  }
]
```



```
}  
]
```

GET /tariffs/{tariffId}

Response Payload

```
{  
  "tariffId": "T101",  
  "utilityId": "U1",  
  "slabs": [  
    {"min": 0, "max": 100, "rate": 3.5},  
    {"min": 101, "max": 300, "rate": 5.0}  
  ]  
}
```

GET /tariffs/utility/{utilityId}

Response Payload

```
{  
  "tariffId": "T101",  
  "utilityId": "U1",  
  "fixedCharge": 120,  
  "taxPercent": 5  
}
```

PUT /tariffs/{tariffId}

Request Payload

```
{  
  "fixedCharge": 150,  
  "taxPercent": 6  
}
```

Response Payload

```
{  
  "message": "Tariff updated successfully"  
}
```

Database

- utilities collection
- tariff_plans collection

6.4 Meter Service

Responsibilities

- Record monthly meter readings
- Validate readings against previous values
- Calculate units consumed per billing cycle

APIs

POST /meter-readings

Request Payload

```
{  
  "consumerId": "C201",  
  "utilityId": "U1",  
  "previousReading": 330,  
  "currentReading": 450,  
  "readingDate": "2025-03-01"  
}
```

Response Payload

```
{
  "meterReadingId":"MR401",
  "unitsConsumed":120,
  "message":"Meter reading recorded successfully"
}
```

GET /meter-readings/{consumerId}

Response Payload

```
[
  {
    "meterReadingId":"MR401",
    "utilityId":"U1",
    "previousReading":330,
    "currentReading":450,
    "unitsConsumed":120,
    "readingDate":"2025-03-01"
  }
]
```

GET /meter-readings/latest/{consumerId}

Response Payload

```
{
  "meterReadingId":"MR401",
  "utilityId":"U1",
  "currentReading":450,
  "unitsConsumed":120,
  "readingDate":"2025-03-01"
}
```

Database

- meter_readings collection
-

6.5 Billing Service

Responsibilities

- Generate bills based on meter readings and tariffs
- Apply fixed charges and taxes
- Maintain bill status transitions

APIs

POST /bills/generate

Request Payload

```
{
  "consumerId":"C201",
  "utilityId":"U1",
  "billingMonth":"MARCH",
  "billingYear":2025
}
```

Response Payload

```
{
  "billId":"B501",
  "totalUnits":120,
  "energyCharge":950,
  "fixedCharge":120,
}
```

```
"taxAmount":45,  
"totalAmount":1115,  
"status":"GENERATED"  
}
```

GET /bills/{billId}

Response Payload

```
{  
  "billId":"B501",  
  "consumerId":"C201",  
  "utilityId":"U1",  
  "billingPeriod":"MARCH-2025",  
  "totalAmount":1115,  
  "status":"DUE"  
}
```

GET /bills/consumer/{consumerId}

Response Payload

```
[  
  {  
    "billId":"B501",  
    "billingPeriod":"MARCH-2025",  
    "totalAmount":1115,  
    "status":"DUE"  
  }  
]
```

GET /bills

Response Payload

```
[  
  {  
    "billId":"B501",  
    "consumerId":"C201",  
    "totalAmount":1115,  
    "status":"DUE"  
  }  
]
```

PUT /bills/{billId}/status

Request Payload

```
{  
  "status":"DUE"  
}
```

Response Payload

```
{  
  "message":"Bill marked as PAID"  
}
```

Database

- bills collection

6.6 Notification Service

Responsibilities

- Send bill generation notifications
- Send payment reminders

- Send password reset notifications

APIs

POST /notifications/bill-generated

Request Payload

```
{
  "consumerId": "C201",
  "billId": "B501",
  "message": "Your electricity bill for March 2025 has been generated."
}
```

Response Payload

```
{
  "notificationId": "N601",
  "status": "SENT"
}
```

POST /notifications/payment-reminder

Request Payload

```
{
  "consumerId": "C201",
  "billId": "B501",
  "dueDate": "2025-03-15"
}
```

Response Payload

```
{
  "notificationId": "N602",
  "status": "SENT"
}
```

POST /notifications/password-reset

Request Payload

```
{
  "email": "john.doe@gmail.com",
  "resetLink": "https://app/reset-password?token=abc123"
}
```

Response Payload

```
{
  "email": "john.doe@gmail.com",
  "resetLink": "https://app/reset-password?token=abc123"
}
```

GET /notifications/{consumerId}

Response Payload

```
[
  {
    "notificationId": "N601",
    "type": "BILL_GENERATED",
    "message": "Your electricity bill has been generated.",
    "status": "SENT"
  }
]
```

Database

- notifications collection

7. DATA DESIGN (LLD)

User DB

User Document

```
{
  "_id": ObjectId,
  "username": "string",
  "email": "string",
  "password": "string (encrypted)",
  "roles": ["ADMIN | USER | ACCOUNTS_OFFICER | BILLING_OFFICER"],
  "enabled": true,
  "passwordUpdatedAt": ISODate,
  "createdAt": ISODate
}
```

Consumers DB

Consumer Document

```
{
  "_id": ObjectId,
  "fullName": "string",
  "email": "string",
  "phone": "string",
  "addressLine1": "string",
  "city": "string",
  "state": "string",
  "postalCode": "string",
  "active": true,
  "createdAt": ISODate,
  "updatedAt": ISODate
}
```

Consumer Request Document

```
{
  "_id": ObjectId,
  "fullName": "string",
  "email": "string",
  "phone": "string",
  "addressLine1": "string",
  "city": "string",
  "state": "string",
  "postalCode": "string",
  "status": "PENDING | APPROVED | REJECTED",
  "createdAt": ISODate,
  "updatedAt": ISODate
}
```

Meter DB

Connection Request Document

```
{
  "_id": ObjectId,
  "consumerId": "ObjectId",
  "utilityType": "ELECTRICITY | GAS | WATER",
  "tariffPlan": "DOMESTIC | COMMERCIAL",
  "status": "PENDING | APPROVED | REJECTED",
  "createdAt": ISODate
}
```

Meter Document

```
{
  "_id": ObjectId,
  "meterNumber": "string",
  "consumerId": "ObjectId",
}
```

```

    "utilityType": "ELECTRICITY | GAS | WATER",
    "tariffPlan": "DOMESTIC | COMMERCIAL",
    "lastReading": "number",
    "installationDate": ISODate,
    "active": true
  }
}

```

Meter Reading Document

```

{
  "_id": ObjectId,
  "meterNumber": "string",
  "readingValue": "number",
  "readingDate": ISODate
}

```

Tariff DB

Tariff Document

```

{
  "_id": ObjectId,
  "utilityType": "ELECTRICITY | GAS | WATER",
  "plans": [
    {
      "plan": "DOMESTIC | COMMERCIAL",
      "active": true,
      "fixedCharge": "number",
      "taxPercentage": "number",
      "slabs": [
        {
          "fromUnit": "number",
          "toUnit": "number",
          "ratePerUnit": "number"
        }
      ],
      "overduePenaltySlabs": [
        {
          "fromDay": "number",
          "toDay": "number",
          "penaltyPercentage": "number"
        }
      ]
    }
  ],
  "effectiveFrom": ISODate
}

```

Bills & Payments DB

Bill Document

```

{
  "_id": ObjectId,
  "consumerId": "ObjectId",
  "meterNumber": "string",
  "utilityType": "ELECTRICITY | GAS | WATER",
  "tariffPlan": "DOMESTIC | COMMERCIAL",
  "previousReading": "number",
  "currentReading": "number",
  "unitsConsumed": "number",
  "energyCharge": "number",
  "fixedCharge": "number",
  "taxAmount": "number",
  "penaltyAmount": "number",
  "totalAmount": "number",
  "status": "DUE | PAID | OVERDUE",
}

```

```

    "generatedAt": ISODate,
    "dueDate": ISODate,
    "lastUpdatedAt": ISODate
}

```

Payment Document

```

{
  "_id": ObjectId,
  "billId": "ObjectId",
  "consumerId": "ObjectId",
  "utilityType": "ELECTRICITY | GAS | WATER",
  "amount": "number",
  "mode": "ONLINE | OFFLINE",
  "status": "INITIATED | SUCCESS | FAILED",
  "processedBy": "SYSTEM | ACCOUNTS_OFFICER",
  "initiatedAt": ISODate,
  "completedAt": ISODate
}

```

Invoice Document

```

{
  "_id": ObjectId,
  "paymentId": "ObjectId",
  "consumerId": "ObjectId",
  "meterNumber": "string",
  "utilityType": "ELECTRICITY | GAS | WATER",
  "previousReading": "number",
  "currentReading": "number",
  "unitsConsumed": "number",
  "energyCharge": "number",
  "fixedCharge": "number",
  "taxAmount": "number",
  "penaltyAmount": "number",
  "totalAmount": "number",
  "paymentMode": "ONLINE | OFFLINE",
  "paymentDate": ISODate,
  "billGeneratedAt": ISODate,
  "billDueDate": ISODate,
  "invoiceGeneratedAt": ISODate
}

```

Notification DB

Notification document

```

{
  "_id": ObjectId,
  "recipient": "string (email)",
  "type": "PASSWORD_RESET | CONSUMER_APPROVED | CONNECTION_APPROVED |
BILL GENERATED | PAYMENT_OTP | PAYMENT_SUCCESS | PAYMENT_FAILED |
INVOICE_PDF",
  "channel": "EMAIL",
  "status": "SENT | FAILED",
  "subject": "string",
  "body": "string",
  "createdAt": ISODate
}

```

8. API DESIGN & VALIDATION

- RESTful principles

- JSON request/response
 - Bean Validation at DTO layer
 - Service-level business rule enforcement
 - Standard HTTP status codes
-

9. ERROR HANDLING STRATEGY

Global Exception Handling

- Centralized using `@ControllerAdvice`
- Standard error response format

```
{
  "timestamp": "2025-01-01T10:00:00",
  "status": 400,
  "error": "Validation Error",
  "message": "Price must be greater than zero"
}
```

10. SECURITY DESIGN

- Password encryption (BCrypt)
 - Role-based access control
 - JWT authentication (optional enhancement)
 - Secure API access
-

11. Business Rules

1. Authentication & Authorization Service

- Users must authenticate using valid credentials.
- JWT tokens are issued after successful authentication.
- Tokens contain user identity and role information.
- Role-based access control is enforced across all services.
- Only active users are allowed to access the system.
- Password reset tokens are single-use and time-bound.

2. Consumer Service

- Each consumer must be linked to a valid authenticated user.
- A consumer can have multiple utility connections.
- Consumer profile data can be modified only by authorized users.
- Consumer records are soft-deleted to preserve historical data.
- A consumer request must be approved before any utility connection can be requested.
- Rejected consumer requests cannot be reused and must be re-submitted.

3. Utility Connection Service

- Each utility connection must have a defined utility type and tariff plan.
- A connection belongs to exactly one consumer.
- Tariff changes do not affect previously generated bills.
- A consumer cannot have more than one active connection of the same utility type.

4. Meter Reading Service

- Meter readings can be recorded only for active connections.
- New readings must not be less than previous readings.
- Only authorized role(billing officer) can submit meter readings.
- Meter readings are immutable once recorded.

5. Billing Service

- Bills are generated based on meter readings and tariff plans.
- Each bill has a defined lifecycle: Due, Paid, or Overdue.
- Bills marked as paid cannot be modified.
- Overdue status is determined based on due date.
- Penalty calculation applies only after due date is crossed.

6. Payment Service

- Online payments require OTP verification.
- Payment is confirmed only after successful OTP validation.
- Offline payments can be recorded only by authorized staff(accounts officer).
- A bill is marked as paid only after successful payment confirmation.
- Each successful payment generates an invoice.

7. Notification Service

- Notifications are sent for bill generation and payment events.
- Notification failures do not impact core business flow.
- Each notification is associated with a specific user and event.

8. Dashboard & Reporting Service

- Dashboard data is read-only and aggregated from multiple services.
- Financial reports are accessible only to authorized roles.
- Reports reflect consistent and finalized transactional data.

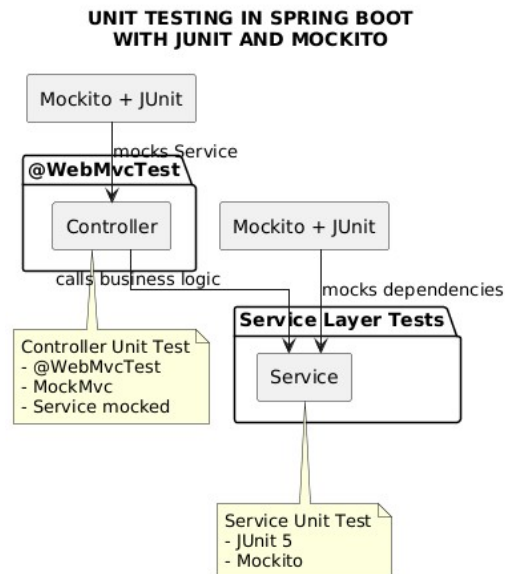
9. System-Wide Rules

- All inter-service communication is secured using JWT.
- Services are loosely coupled and independently deployable.
- Circuit breakers handle service failures gracefully.

11. NON-FUNCTIONAL REQUIREMENTS

Area	Design Decision
Scalability	Stateless services, Docker
Performance	Pagination, async calls
Availability	Independent services
Maintainability	POM-like layered backend
Security	Validation, encryption

12. TESTING STRATEGY

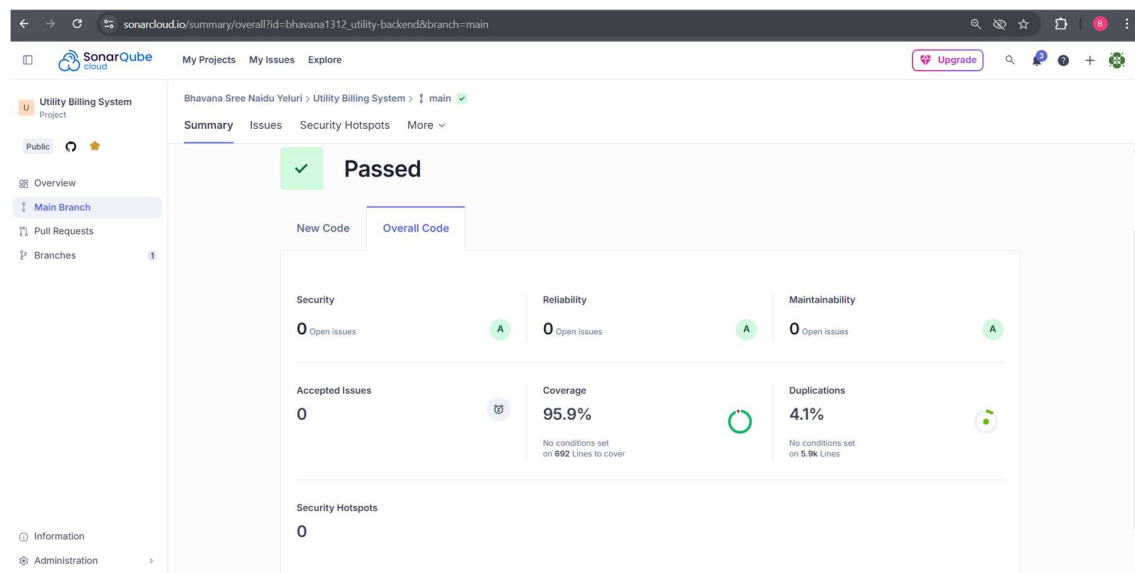


Backend

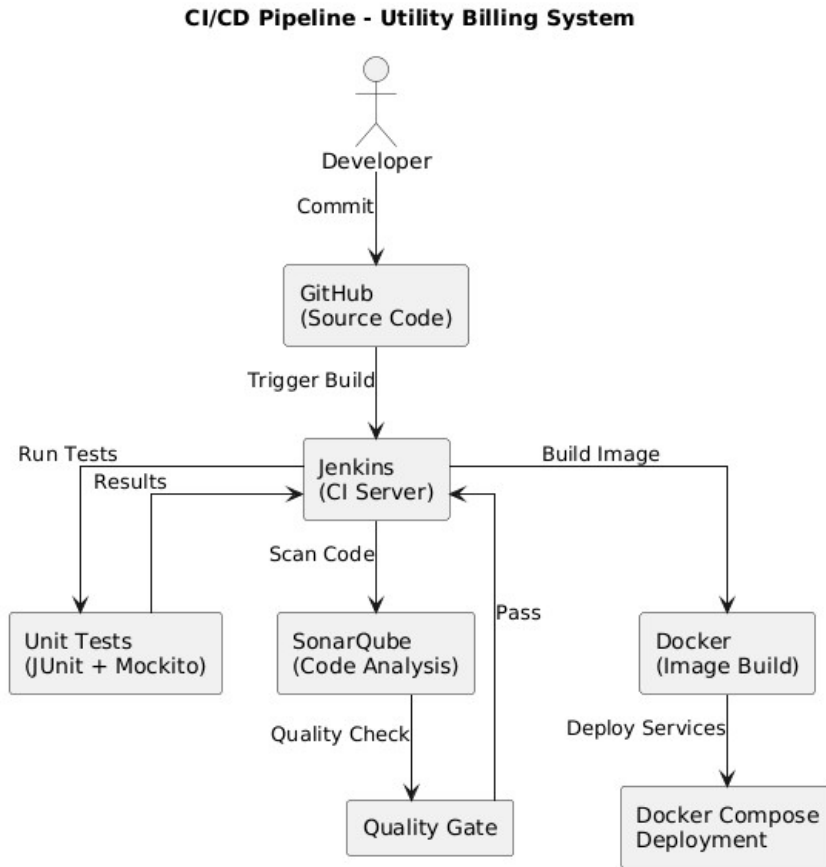
- Unit tests (Service layer)
- Controller tests
- 95.9% coverage

Quality Gates

- SonarQube enforced
- Build fails on violations



13. CI/CD DESIGN



Pipeline Flow

1. Git Commit
2. Jenkins Build
3. Unit Tests
4. SonarQube Scan
5. Quality Gate Check
6. Docker Build
7. Docker Compose Deploy

14. DEPLOYMENT DESIGN

- Docker image per microservice
 - docker-compose for orchestration
 - Environment-specific configs
-

15. ASSUMPTIONS & CONSTRAINTS

Assumptions

- Services communicate over REST
- MongoDB available
- Docker environment present

Constraints

- No distributed transactions
 - Event-driven architecture out of scope
-

16. FUTURE ENHANCEMENTS

- Kubernetes deployment
 - Centralized logging (ELK)
-

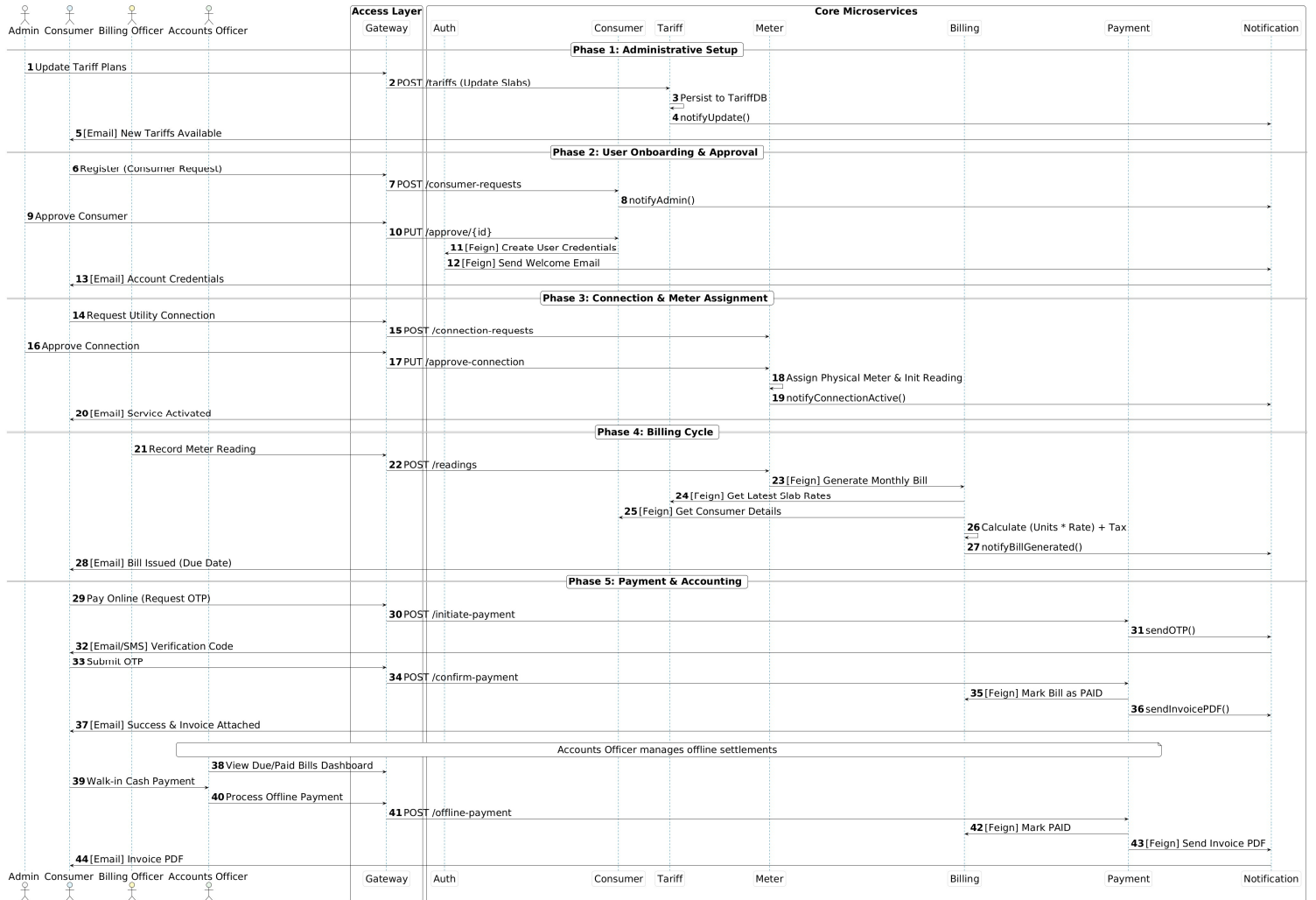
17. CONCLUSION

This design ensures:

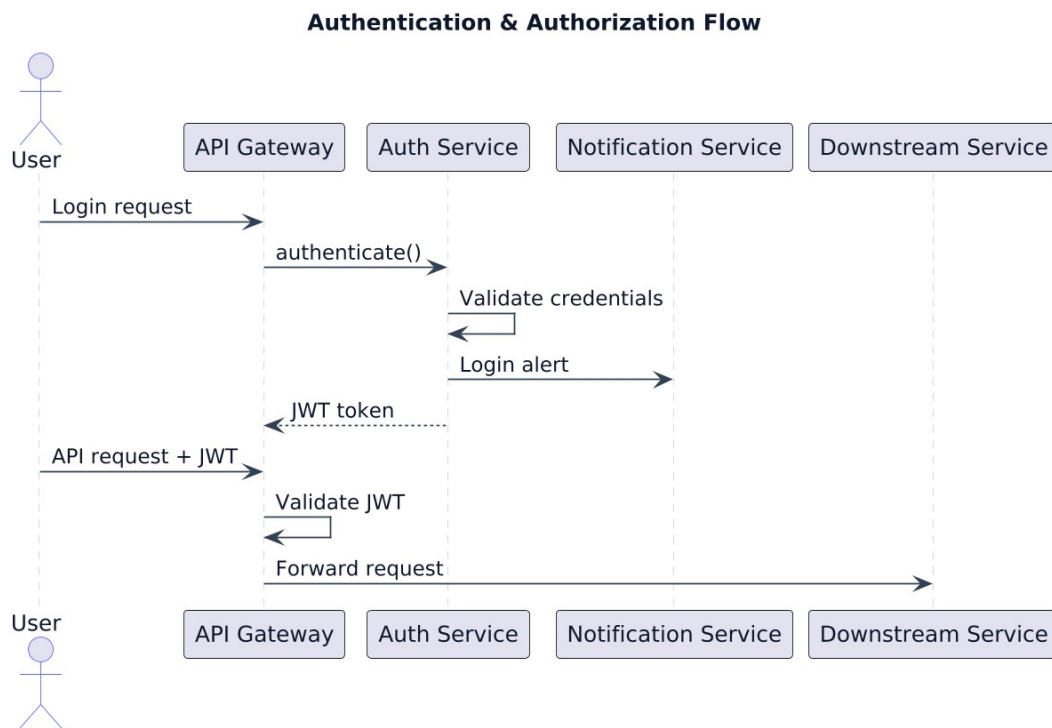
- ✓ Clean separation of concerns
- ✓ Scalability & maintainability
- ✓ Testability & CI/CD readiness
- ✓ Interview-ready explanation

SEQUENCE DIAGRAMS

UTILITY BILLING SYSTEM



1. Authentication & Authorization — Sequence Diagram



Scenario

A user authenticates using valid credentials, receives a JWT token, and accesses secured backend services through the API Gateway.

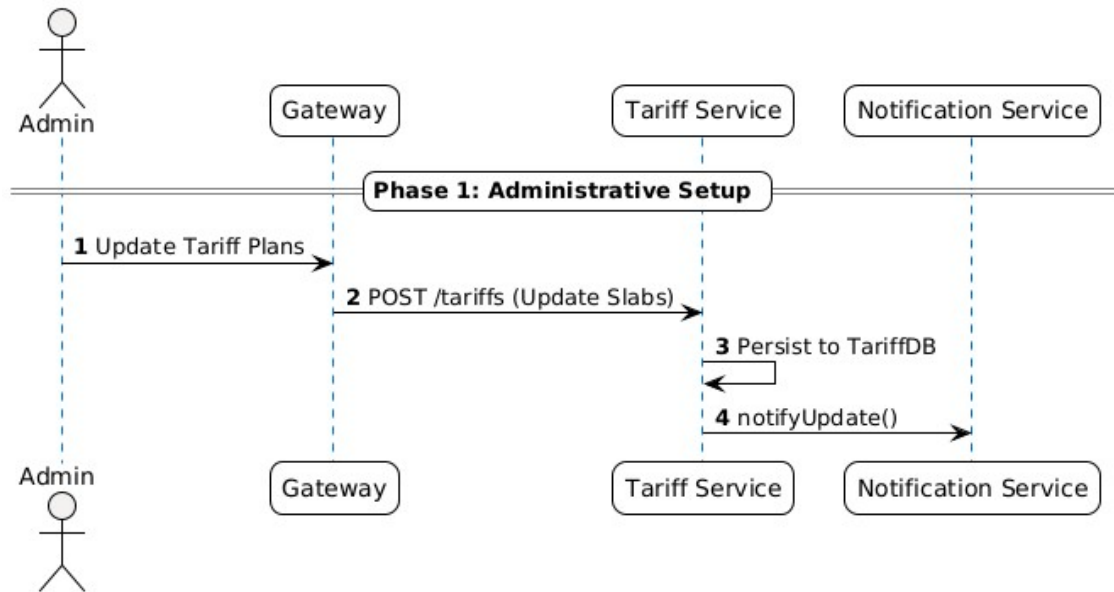
Flow

1. The user submits login credentials to the API Gateway.
2. The API Gateway forwards the authentication request to the Auth Service.
3. The Auth Service validates the user credentials against stored user data.
4. Upon successful authentication, the Auth Service generates a JWT token.
5. A login notification is sent via the Notification Service.
6. The JWT token is returned to the user through the API Gateway.
7. For subsequent requests, the user sends the JWT token along with the API request.
8. The API Gateway validates the JWT token and extracts user role information.
9. If valid, the request is forwarded to the appropriate downstream service.

Participants

- User
 - API Gateway
 - Auth Service
 - Notification Service
 - Downstream Service
-

2. Administrative Setup— Sequence Diagram



Scenario

The administrator configures or updates tariff plans, which are stored centrally.

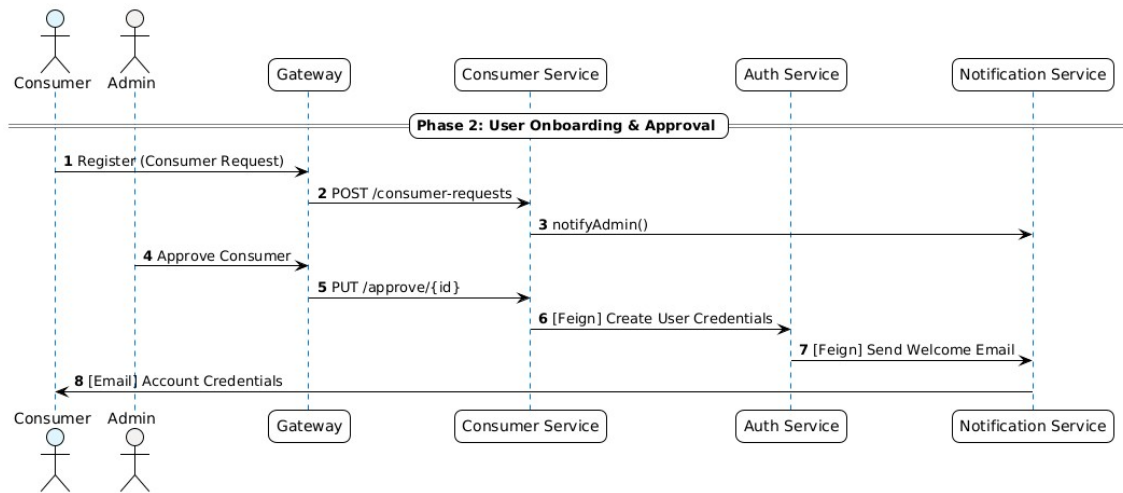
Flow

1. Admin updates tariff plans through the Gateway.
2. Gateway forwards the request to the Tariff Service.
3. Tariff Service saves the updated tariff configuration.

Participants

- Admin
 - API Gateway
 - Tariff Service
 - Notification Service
 - Consumer
-

3. User Onboarding & Approval — Sequence Diagram



Scenario

A consumer registers in the system and gets approved by the administrator to gain portal access.

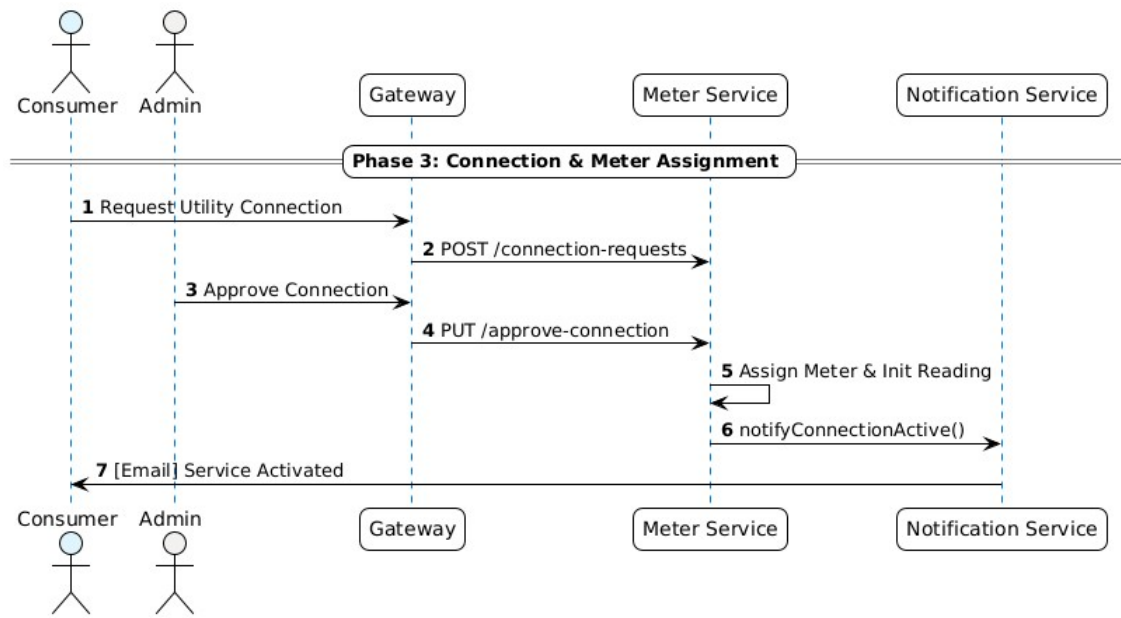
Flow

1. Consumer submits a registration request through the Gateway.
2. Consumer Service creates a consumer request.
3. Admin approves the consumer request.
4. Auth Service creates login credentials.
5. Credentials are sent to the consumer via email.

Participants

- Consumer
 - Admin
 - API Gateway
 - Consumer Service
 - Auth Service
 - Notification Service
-

4. Connection & Meter Assignment — Sequence Diagram



Scenario

An approved consumer requests a utility connection, which is validated and assigned a meter by the administrator.

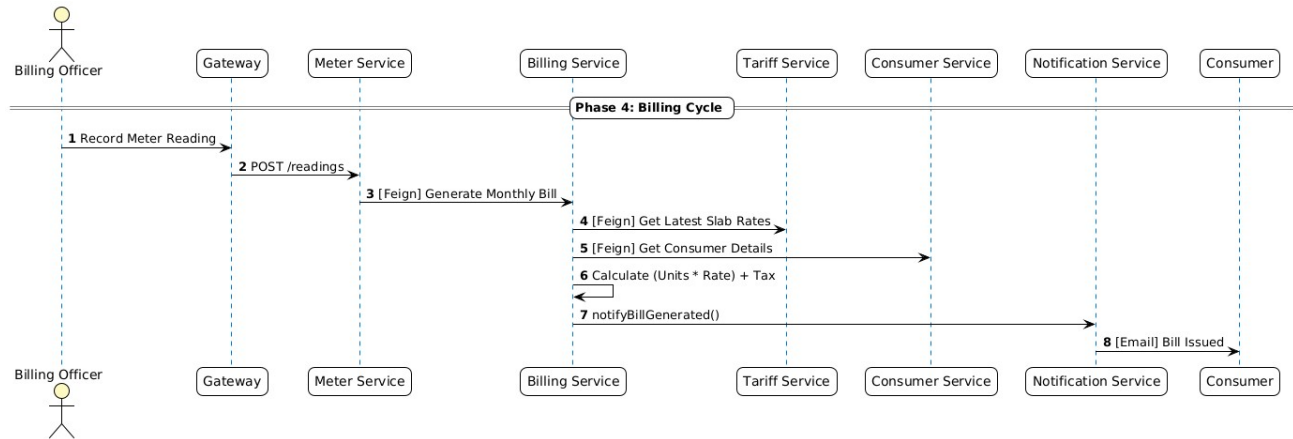
Flow

1. Consumer requests a utility connection.
2. Meter Service records the connection request.
3. Admin approves the connection request.
4. A meter is assigned and initialized.
5. Consumer is notified about service activation.

Participants

- Consumer
 - Admin
 - API Gateway
 - Meter Service
 - Notification Service
-

5. Billing Cycle — Sequence Diagram



Scenario

Periodic meter readings are recorded and used to generate utility bills for consumers.

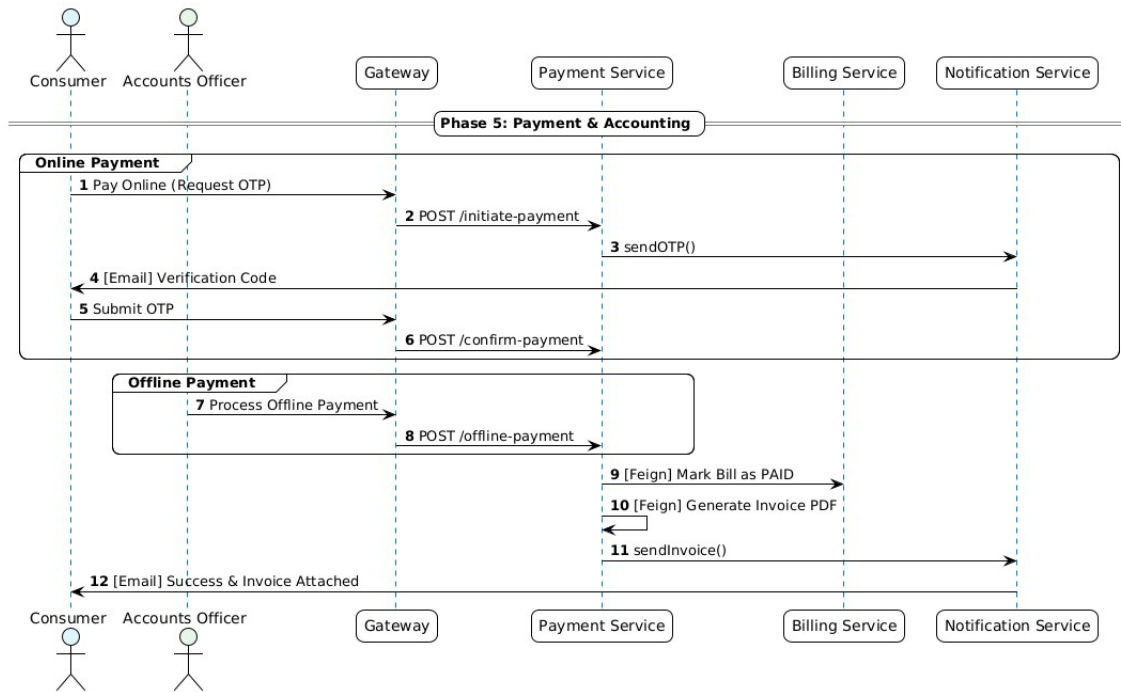
Flow

1. Billing Officer records meter readings.
2. Meter Service validates and forwards data for billing.
3. Billing Service retrieves tariff and consumer details.
4. Bill amount is calculated based on consumption and slabs.
5. Bill is generated and notified to the consumer.

Participants

- Billing Officer
 - API Gateway
 - Meter Service
 - Billing Service
 - Tariff Service
 - Consumer Service
 - Notification Service
-

6. Payment & Accounting — Sequence Diagram



Scenario

Consumers settle bills through online or offline payment methods, and invoices are generated.

Flow

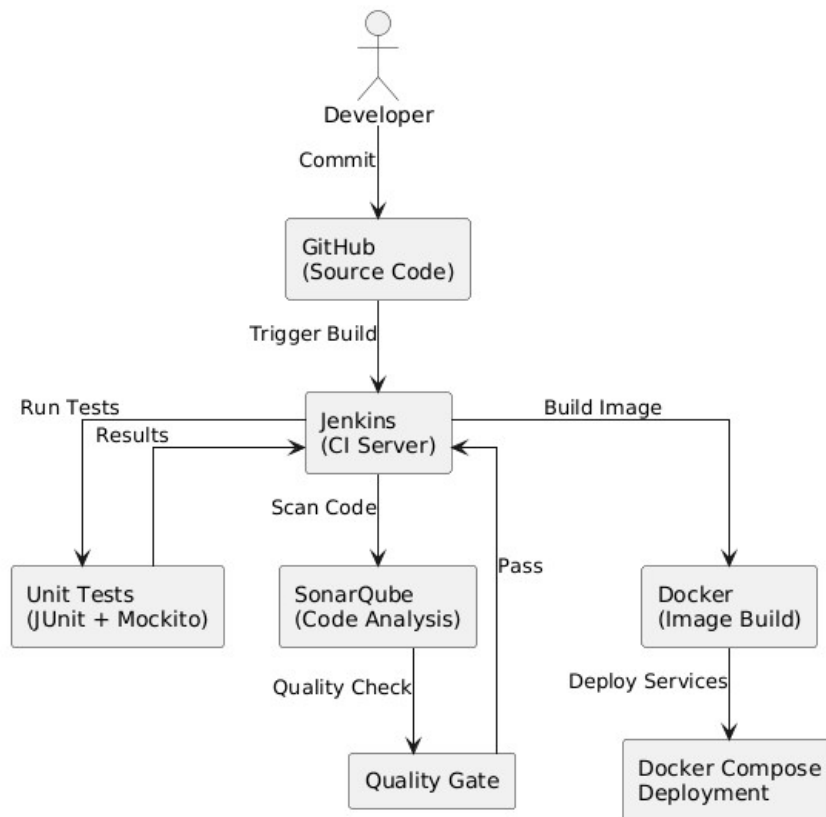
1. Consumer initiates online payment and requests OTP.
2. OTP is sent and verified before confirmation.
3. Accounts Officer can record offline payments.
4. Payment Service marks the bill as paid.
5. Invoice is generated and emailed to the consumer.

Participants

- Consumer
- Accounts Officer
- API Gateway
- Payment Service
- Billing Service
- Notification Service

7. CI/CD — Sequence Diagram

CI/CD Pipeline - Utility Billing System



Scenario

Developer pushes code to Git repository.

Flow

1. Developer pushes code
2. Jenkins pipeline triggered
3. Jenkins runs unit tests
4. Jenkins runs SonarQube scan
5. Quality gate checked
6. Docker images built
7. Docker Compose deploys services

Participants

- Developer
 - Git
 - Jenkins
 - SonarQube
 - Docker
-

7. Error Handling — Sequence Diagram

Scenario

Invalid request sent to backend.

Flow

1. Angular sends invalid request
2. Controller validation fails
3. Global exception handler triggered
4. Standard error response returned

Key Point

- Consistent error structure across services
-